4.1 Sequence models

- we need statistical tools and new deep neural network architectures to deal with sequence data
- to keep things simple, we use the stock price (FTSE 100 index) illustrated in Figure 1, as an example

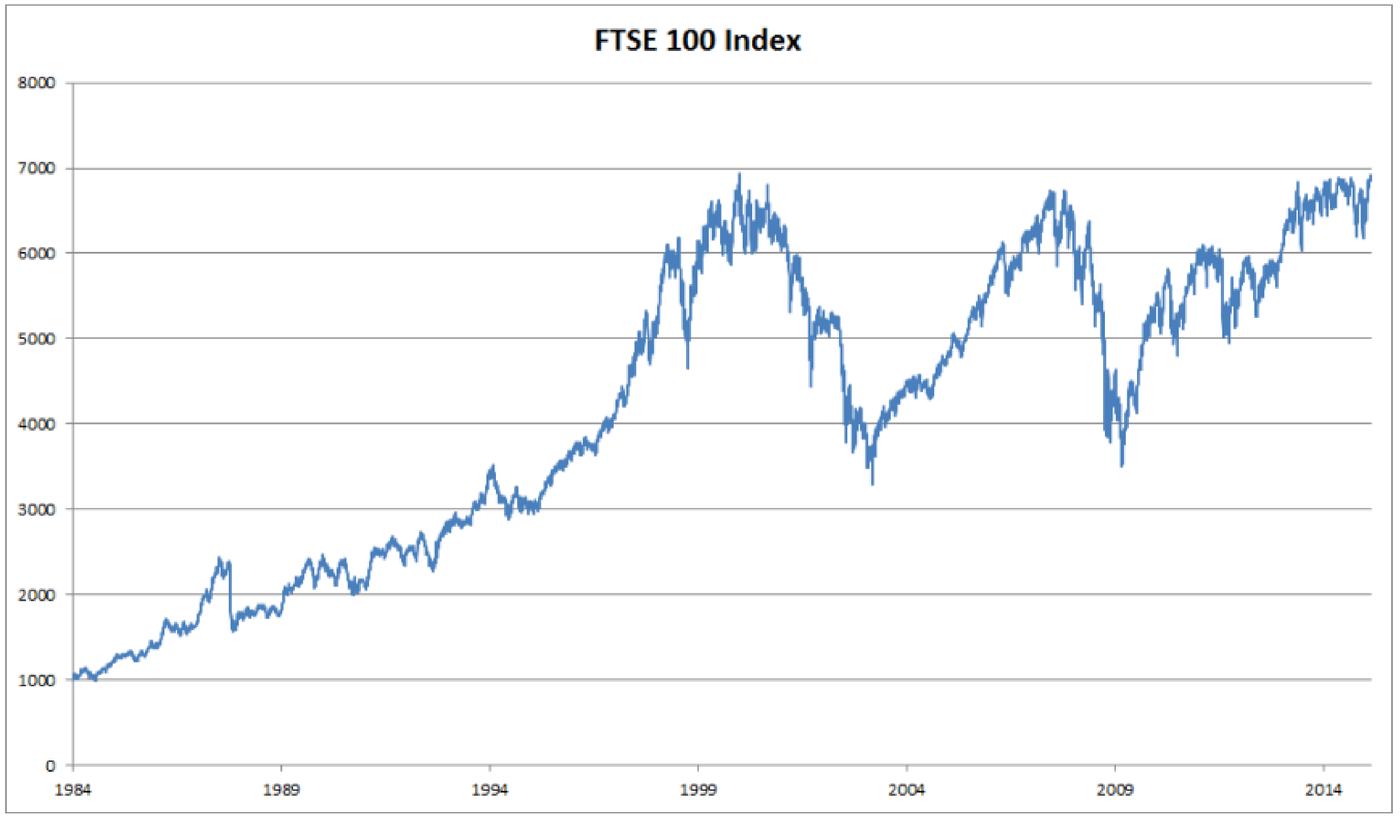


Figure 1: FTSE 100 index over about 30 years.

4.1 Sequence models

- let us denote the prices by x_t, i.e., at *time step* $t \in \mathbb{Z}^+$ we observe price x_t
- note that, for sequences in this chapter, t will typically be discrete and vary over integers or its subset
- suppose that a trader who wants to do well in the stock market on day t predicts x_t via:

$$x_t \sim P(x_t | x_{t-1}, \ldots, x_1).$$

- in order to achieve this, our trader could use a regression model such as the one that we discussed in Chapter 1

4.1 Sequence models

- there is just one major problem: the number of inputs $x_{t-1}, \ldots, x_1$ varies, depending on t
- that is to say, the number increases with the amount of data that we encounter, and we will need an approximation to make this computationally tractable
- much of what follows in this chapter will revolve around how to estimate $P(x_t|x_{t-1}, \ldots, x_1)$ efficiently
- in a nutshell, it boils down to two strategies, as follows

4.1 Sequence models

- first, assume that the potentially rather long sequence $x_{t-1}, \ldots, x_1$ is not really necessary
- in this case, it might be sufficient to use some timespan of length τ, and only use the $x_{t-1}, \ldots, x_{t-\tau}$ observations
- the immediate benefit is that, now, the number of arguments is always the same, at least for $t > \tau$
- this allows us to train a deep network, as indicated above
- such models will be called *autoregressive models*, as they literally perform regression on themselves

4.1 Sequence models

- the second strategy, shown in Figure 2, is to keep some summary h_t of the past observations, and, at the same time, update h_t in addition to the prediction $\hat{x}_t$
- this leads to models that estimate x_t with $\hat{x}_t = P(x_t|h_t)$ and, moreover, updates of the form $h_t = g(h_{t-1}, x_{t-1})$
- since h_t is not observed in the data, these models are also called *latent autoregressive models*

4.1 Sequence models

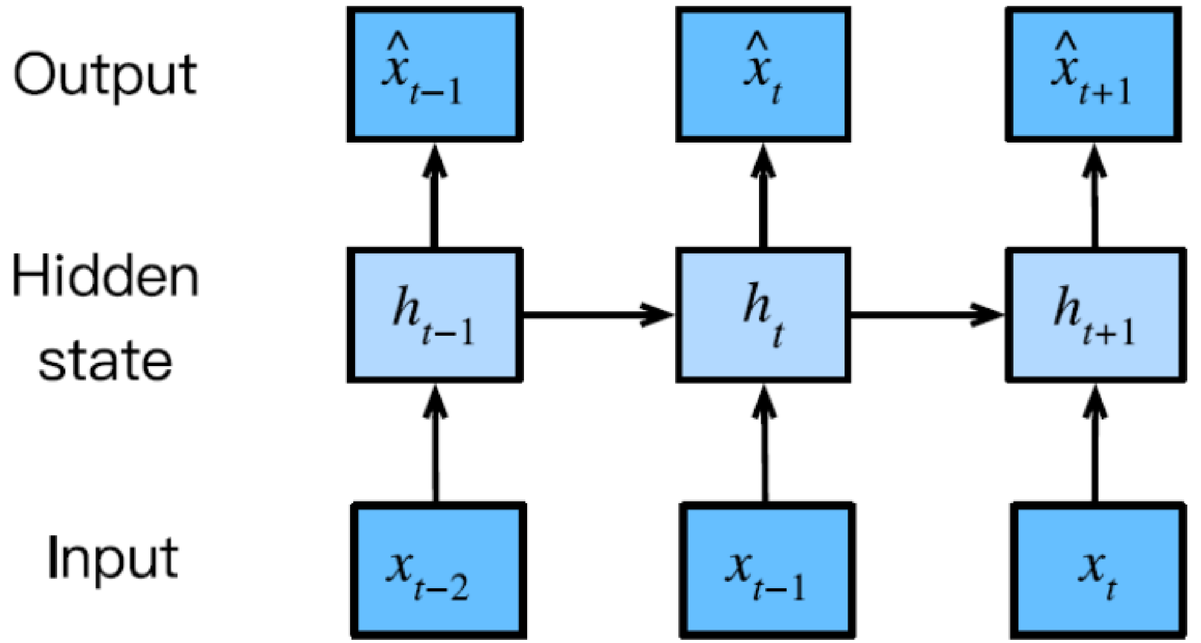


Figure 2: A latent autoregressive model.

4.1 Sequence models

- both cases raise the obvious question of how to generate training data
- we typically use historical observations to predict the next observation, given the ones up to right now
- obviously, we do not expect time to stand still; however, a common assumption is that, while the specific values of x_t might change, at least the dynamics of the sequence itself will not
- this is reasonable, since novel dynamics are just that, novel, and thus not predictable using data that we have so far
- statisticians call dynamics that do not change *stationary*
- regardless of what we do, we will thus get an estimate of the entire sequence via:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t | x_{t-1}, \ldots, x_1).$$

4.1 Sequence models

- note that the above considerations still hold if we deal with discrete objects, such as words, rather than continuous numbers
- the only difference is that, in such a situation, we need to use a classifier rather than a regression model to estimate $P(x_t|x_{t-1}, \ldots, x_1)$
- recall the approximation that, in an autoregressive model, we use only $x_{t-1}, \ldots, x_{t-\tau}$ instead of $x_{t-1}, \ldots, x_1$ to estimate x_t
- whenever this approximation is accurate, we say that the sequence satisfies the *Markov condition*
- in particular, if $\tau = 1$, we have a *first-order Markov model*, and $P(x)$ is given by:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t|x_{t-1}) \text{ where } P(x_1|x_0) = P(x_1).$$

4.1 Sequence models

- such models are particularly useful whenever x_t assumes only a discrete value, since, in this case, dynamic programming can be used to compute values along the chain exactly
- for instance, we can compute $P(x_{t+1}|x_{t-1})$ efficiently:

$$\begin{aligned} P(x_{t+1}|x_{t-1}) &= \frac{\sum_{x_t} P(x_{t+1}, x_t, x_{t-1})}{P(x_{t-1})} \\ &= \frac{\sum_{x_t} P(x_{t+1}|x_t, x_{t-1}) P(x_t, x_{t-1})}{P(x_{t-1})} \\ &= \sum_{x_t} P(x_{t+1}|x_t) P(x_t|x_{t-1}), \end{aligned}$$

  by using the fact that we only need to take into account a very short history of past observations: $P(x_{t+1}|x_t, x_{t-1}) = P(x_{t+1}|x_t)$
- going into details of dynamic programming is beyond the scope of this section
- control and reinforcement learning algorithms use such tools extensively

4.1 Sequence models

- in principle, there is nothing wrong with unfolding $P(x_1, \ldots, x_T)$ in reverse order
- after all, by conditioning, we can always write it via:

$$P(x_1, \ldots, x_T) = \prod_{t=T}^{1} P(x_t | x_{t+1}, \ldots, x_T).$$

- in fact, if we have a Markov model, we can obtain a reverse conditional probability distribution, too
- in many cases, however, there exists a natural direction for the data, namely going forward in time
- it is clear that future events cannot influence the past

4.1 Sequence models

- hence, if we change x_t, we may be able to influence what happens for x_{t+1} going forward, but not the converse
- that is, if we change x_t, the distribution over past events will not change
- consequently, it should to be easier to explain $P(x_{t+1}|x_t)$ rather than $P(x_t|x_{t+1})$
- for instance, it has been shown that, in some cases, we can find $x_{t+1} = f(x_t) + \epsilon$ for some additive noise ϵ, whereas the converse is not true
- this is great news, since it is typically the forward direction that we are interested in estimating

4.1 Sequence models

- we have reviewed statistical tools and prediction challenges for sequence data; such data can take many forms
- specifically, text is one of the most popular examples of sequence data
- for example, an article can be simply viewed as a sequence of words, or even a sequence of characters
- to facilitate our future experiments with sequence data, we will explain common preprocessing steps for text
- usually, these steps are:
  1. Load text as strings into memory.
  2. Split strings into *tokens* (e.g., words and characters).
  3. Build a *table* of vocabulary to map the split tokens to numerical indices.
  4. Convert text into sequences of numerical indices so they can be manipulated by models easily.

4.1 Sequence models

- each text sequence is split into a list of tokens
- a *token* is the basic unit in text; in the end, a list of token lists are obtained, where each token is a string
- the string type of the token is inconvenient to be used by models, which take numerical inputs
- now, we build a dictionary, often called a *vocabulary* as well, to map string tokens into numerical indices starting from 0

4.1 Sequence models

- to do so, we first count the unique tokens in all the documents from the training set, namely a *corpus*, and then assign a numerical index to each unique token according to its frequency
- rarely appeared tokens are often removed to reduce the complexity
- any token that does not exist in the corpus, or has been removed, is mapped into a special unknown token “<unk>”
- we can also add a list of reserved tokens, such as “<pad>” for padding, “<bos>” to represent the beginning of a sequence, and “<eos>” for the end of a sequence

4.1 Sequence models

- so, we saw how to map text data into tokens, where these tokens can be viewed as a sequence of discrete observations, such as words or characters
- assume that the tokens in a text sequence of length T are $x_1, x_2, \ldots, x_T$
- then, in the text sequence, x_t $(1 \leq t \leq T)$ can be considered as the observation or label at time step t
- given such a text sequence, the goal of a *language model* is to estimate the joint probability of the sequence $P(x_1, x_2, \ldots, x_T)$

4.1 Sequence models

- language models are incredibly useful
- for instance, an ideal language model would be able to generate natural text just on its own, simply by drawing one token at a time $x_t \sim P(x_t|x_{t-1}, \ldots, x_1)$
- all text emerging from such a model would pass as natural language, e.g., English text
- furthermore, it would be sufficient for generating a meaningful dialog, simply by conditioning the text on previous dialog fragments
- clearly, we are still very far from designing such a system, since it would need to *understand* the text, rather than just generate grammatically correct content

4.1 Sequence models

- let us now apply Markov models to language modeling
- a distribution over sequences satisfies the Markov property of first order if $P(x_{t+1}|x_t, \ldots, x_1) = P(x_{t+1}|x_t)$
- higher orders correspond to longer dependencies
- this leads to a number of approximations that we could apply to model a sequence:

$$\begin{aligned} P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2)P(x_3)P(x_4), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2|x_1)P(x_3|x_2)P(x_4|x_3), \\ P(x_1, x_2, x_3, x_4) &= P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)P(x_4|x_2, x_3). \end{aligned}$$

- the probability formulas that involve one, two, three, and, in general, n variables are typically referred to as *unigram*, *bigram*, *trigram*, and *n-gram* models, respectively